



Exceções



Exceções em Java

São erros que ocorrem em tempo de execução

- Eles ocorrem porque a linha de código a ser executada não pode fazer o escrito por algum motivo
- São erros difíceis de prever durante o desenvolvimento da aplicação.
- Se ocorrerem, o aplicativo trava e fecha sem aviso prévio. Uma vez que eles é difícil prevêê-los, normalmente é simplesmente capturado para que o aplicativo não trave e notifica o usuário de alguma forma
- Exemplos:
 - Durante o download de um arquivo, a conexão com a Internet falha
 - Ao gravar no disco, não temos permissão para fazer isso ou o disco está cheio
 - Acessamos atributos ou métodos de um objeto cujo valor não existe (é nulo)
 - Acessamos uma posição inválida em uma lista
 - Realizamos uma operação matemática inválida (divisão por zero)



Bloco Try-Catch

- Quando o código é suscetível a uma exceção, teremos que incluí-lo em um bloco try-catch:

```
try{  
    <linha que gera a exceção>  
} catch (Exception ex) {  
    // A execução segue por aqui quando a exceção ocorre  
    // No objeto e há informações sobre a exceção  
}
```

Para o exemplo acima:

- `e.printStackTrace()` irá pintar na tela o traço da exceção (arquivos e linha de código onde ela é gerada)
- `e.getMessage()` conterá a mensagem de erro associada à exceção



Instrução throws

- A instrução throws é usada quando o método que lança a exceção não é adequado para capturá-la. Com esta instrução é notificado que este método não pode lançar a exceção, logo será quem o invocou o método, que terá que incluir o bloco try-catch

```
public void aMethod () throws IOException {  
    // Em algum lugar neste método, uma exceção é lançada  
    // do tipo IOException, mas este não é o momento para capturá-lo  
    ...  
    ...  
}
```



Exceções verificadas e não verificadas

- Exceções verificadas:
 - Exceções que devem ser capturadas ou declaradas no método onde ocorrem
 - Herdar diretamente da classe Exception
 - Como regra geral, eles serão o tipo de exceção que devemos sempre capturar em nosso código.
 - IOException e ParseException são dois exemplos de exceções "verificadas"
- Exceções não verificadas:
 - Exceções que podem ser lançadas sem serem detectadas ou declaradas
 - Herdar da classe RuntimeException, que por sua vez herda de Exception
 - Como regra geral, eles serão o tipo de exceção que não devemos capturar. Devemos evitar que ocorram como, por exemplo, no caso da exceção "desmarcada" NullPointerException
 - ArithmeticException e ArrayIndexOutOfBoundsException são outros exemplos de exceções "não verificadas".



Criar novas exceções

- É possível criar novas exceções estendendo a classe Exception.
- Java nunca lançará suas próprias exceções, então use a instrução throw para lançá-la quando for detectada o caso de erro

```
public class UserNaoExisteException throws Exception {  
    public UserNaoExisteException (String mensagem){  
        super (mensagem);  
    }  
}  
...  
if (usuario == null)  
    throw new UserNaoExisteException ("Não se pode acessar o perfil desse usuário, verifique a base de dados");  
...
```



Dúvidas?

Tokio.